```
(gdb) print *arr@10
$6 = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
(gdb) print (int[10])*arr
$7 = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
(gdb) print /x (int[10])*arr
$8 = {0x0, 0x1, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x9}
(gdb) x /10dw arr
0x7fffffffd520: 0       1       2       3
0x7fffffffd530: 4       5       6       7
0x7fffffffd540: 8       9
(gdb)
```

The large hex numbers that "x" prints in the left-most column are the memory addresses of the printed rows

If you want you can override the display format by specifying the format with print as print /format where format is x for hex, d for signed decimal, o for octal, s for string. There are more format options available.

Another command to display the value of variables is x for eXplore, which takes a memory location rather than a variable name. It can be quite useful for exploring pointers and arrays. If you provide it an address, or the name of a pointer, it can be used not only to display the value stored at that memory location, but also of consecutive memory locations starting at that location.

The full syntax for the command is x /nfu <addr> where n specifies how many consecutive memory locations to print; f is the format of the data as explained for the print command, and u is the size of each memory location. So x /5dw arr will print 5 decimal (d) words (32bits) starting at the pointer arr.

Finally let's look at the display command. Chances are you want to continuously look at the value of a particular variable or memory location after each step. Rather then using the print or x commands again and again, you can use thedisplay /format <var/address> command. It will automatically display data using the print or x command based on your parameters each time you step forward in your code.

Conclusion

The GNU debugger is a phenomenally powerful tool, even with the basic introductory knowledge that you have gained from this article. There is of course a whole lot more that you can do with gdb.

GDB Shortcuts

Most gdb commands can be abbreviated to just one or two letters. Here are a few you might find useful:

- run – r
- step – s
- stepi – si
- next – n
- nexti – ni
- print – p
- list – l
- continue – c

Often, you need to repeat the same command again and again. For instance, you might want to step through code a couple of times. Rather than type "step" repeatedly, or even type "s" repeatedly, you can just press [Enter] to repeat the previous command.

```
(gdb) step
5           for (i = 0; i <= len; ++i) {
2: sum = -10896
1: i = 0
(gdb) step
6               sum += arr[i];
2: sum = -10896
1: i = 1
(gdb) step
5           for (i = 0; i <= len; ++i) {
2: sum = -10895
1: i = 1
(gdb) step
6               sum += arr[i];
2: sum = -10895
1: i = 2
(gdb) step
5           for (i = 0; i <= len; ++i) {
2: sum = -10893
1: i = 2
(gdb)
```

With each step here gdb is displaying both, the value of i the loop counter and sum

For instance, gdb can also be used to debug a program that is already running, and in fact it can even connect to a remote computer and debug programs running on it. It also supports reverse debugging. It can even record and play back the execution of applications and a whole lot more. We've only scratched the surface here but hopefully we have provided you with a good launching pad. >

*Fun Facts

*Old is still Gold

>>A list of programming languages, some of which go as far back as 1950s, which are still used in one form or the other. Maybe old is gold, after all? Hit the link below to find out what these languages are...

http://dgit.in/isgoldold

*A brief and absurd history of programming languages

>>Read this man's blog as he gives a "Brief, Incomplete, and Mostly Wrong" History of Programming Languages.

http://dgit.in/historypl

*An emoji programming language

>>Someone on 4chan had this absurd idea of creating a programming language out of emojis. And like most absurd ideas on the internet, it is being made.

http://dgit.in/emoprol